6.7 RECURSION

Recursion is an important concept in computer science. Many algorithms can be best described in terms of recursion. This section introduces this powerful tool, and Sec. 6.9 will show how recursion may be implemented by means of stacks.

Suppose P is a procedure containing either a Call statement to itself or a Call statement to a second procedure that may eventually result in a Call statement back to the original procedure P. Then P is called a *recursive procedure*. So that the program will not continue to run indefinitely, a recursive procedure must have the following two properties:

(1) There must be certain criteria, called *base criteria*, for which the procedure does not call itself.

(2) Each time the procedure does call itself (directly or indirectly), it must be closer to the base criteria.

A recursive procedure with these two properties is said to be *well-defined.*

Similarly, a function is said to be *recursively defined* if the function definition refers to itself. Again, in order for the definition not to be circular, it must have the following two properties:

(1) There must be certain arguments, called *base values*, for which the function does not refer to itself.

(2) Each time the function does refer to itself, the argument of the function must be closer to a base value

A recursive function with these two properties is also said to be well-defined.

The following examples should help clarify these ideas.

Factorial Function

The product of the positive integers from 1 to n, inclusive, is called "n factorial" and is usually denoted by $n!$:

$$n! = 1 \cdot 2 \cdot 3 \ldots (n-2)(n-1)n$$

It is also convenient to define $0! = 1$, so that the function is defined for all nonnegative integers. Thus we have

$$0! = 1 \quad 1! = 1 \quad 2! = 1 \cdot 2 = 2 \quad 3! = 1 \cdot 2 \cdot 3 = 6 \quad 4! = 1 \cdot 2 \cdot 3 \cdot 4 = 24$$
$$5! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120 \quad 6! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 = 720$$

and so on. Observe that

$$5! = 5 \cdot 4! = 5 \cdot 24 = 120 \quad \text{and} \quad 6! = 6 \cdot 5! = 6 \cdot 120 = 720$$

This is true for every positive integer n; that is,

$$n! = n \cdot (n \cdot 1)!$$

Accordingly, the factorial function may also be defined as follows:

Definition 6.1: (Factorial Function)

(a) If $n = 0$, then $n! = 1$.

(b) If $n > 0$, then $n! = n \cdot (n - 1)!$

Observe that this definition of $n!$ is recursive, since it refers to itself when it uses $(n - 1)!$ However, (a) the value of $n!$ is explicitly given when $n = 0$ (thus 0 is the base value); and (b) the value of $n!$ for arbitrary n is defined in terms of a smaller value of n which is closer to the base value 0. Accordingly, the definition is not circular, or in other words, the procedure is well-defined.

Example 6.9

Let us calculate 4! using the recursive definition. This calculation requires the following nine steps:

```
(1) 4! = 4 · 3!
(2)          3! = 3 · 2!
(3)                   2! = 2 · 1!
(4)                            1! = 1 · 0!
(5)                                     0! = 1
(6)                            1! = 1 · 1  = 1
(7)                   2! = 2 · 1 = 2
(8)          3! = 3 · 2 = 6
(9) 4! = 4 · 6 = 24
```

That is:

Step 1. This defines 4! in terms of 3!, so we must postpone evaluating 4! until we evaluate 3! This postponement is indicated by indenting the next step.

Step 2. Here 3! is defined in terms of 2!, so we must postpone evaluating 3! until we evaluate 2!

Step 3. This defines 2! in terms of 1!

Step 4. This defines 1! in terms of 0!

Step 5. This step can explicitly evaluate 0!, since 0 is the base value of the recursive definition.

Steps 6 to 9. We backtrack, using 0! to find 1!, using 1! to find 2!, using 2! to find 3!, and finally using 3! to find 4! This backtracking is indicated by the "reverse" indention.

Observe that we backtrack in the reverse order of the original postponed evaluations. Recall that this type of postponed processing lends itself to the use of stacks. (See Sec. 6.2.)

The following are two procedures that each calculate n factorial.

Procedure 6.9A: FACTORIAL(FACT, N)

This procedure calculates N! and returns the value in the variable FACT.

1. If N = 0, then: Set FACT := 1, and Return.
2. Set FACT := 1. [Initializes FACT for loop.]
3. Repeat for K = 1 to N.
   Set FACT := K*FACT.
   [End of loop.]
4. Return.

Procedure 6.9B: FACTORIAL(FACT, N)

This procedure calculates N! and returns the value in the variable FACT.

1. If N = 0, then: Set FACT := 1, and Return.
2. Call FACTORIAL(FACT, N – 1).
3. Set FACT := N*FACT.
4. Return.